

# ELI5: A fixed-size answer machine

The Tom Klootwijk bounded-query implementation stores a compact rule for motion and a few fixed correction slots. It answers one object-at-one-time question without walking through every old frame.

| CPU TEST          | CUDA CHECK        | QUERY BOUND      |
|-------------------|-------------------|------------------|
| <b>1 / 1 PASS</b> | <b>4,096 PASS</b> | <b>4 + 1 + 4</b> |

## The five-year-old version

Imagine four labeled drawers for time ranges. To answer a question, the program checks all four drawer labels, uses one motion recipe, checks four tiny correction pockets, and reads the guard sign. The number of checks stays the same even if you imagine many more display frames.

|                                     |                              |                                   |                                  |
|-------------------------------------|------------------------------|-----------------------------------|----------------------------------|
| <b>1. Check 4<br/>segment slots</b> | <b>2. Predict<br/>1 seed</b> | <b>3. Check 4<br/>patch slots</b> | <b>4. Evaluate<br/>the guard</b> |
|-------------------------------------|------------------------------|-----------------------------------|----------------------------------|

At  $t=5$  and  $t=9$  the object follows the base seed. From  $t=10$  onward, one active patch adds  $y$  motion, adds phase, and changes mode from 1 to 2. The plane guard is  $x - 8$ , so its value changes from  $-3$  at  $t=5$  to  $+1$  at  $t=9$ .

**Exact claim: one indexed query is  $O(1)$  under the fixed 4-by-4 profile.**

Important limit:  $q$  queries and  $q$  returned answers still cost  $\Theta(q)$ . The implementation does not make nontrivial runtime become little- $o$  of 1.

# Fresh literal results

Executed locally at 2026-08-22T11:38:42+02:00. The blocks below are the program outputs from this run, reproduced line for line.

## CPU test runner

```
Test project C:/Users/Tom/Documents/theTomKlootwijkManifold/tmp/tkm_cpu
  Start 1: tkm_bounded_query_tests
1/1 Test #1: tkm_bounded_query_tests ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 1

Total Test time (real) =  0.01 sec
```

## Demo executable

```
Tom Klootwijk bounded-query substrate demo
carrier: R^3 x S^1, fixed profile: 4 segments x 4 patch slots

t=5.0000 p=(5.0000,0.0000,0.0000) phase=0.5000 mode=1 segment=0 patch_hits=0 guard=-3.0000
t=9.0000 p=(9.0000,0.0000,0.0000) phase=0.9000 mode=1 segment=0 patch_hits=0 guard=1.0000
t=11.0000 p=(11.0000,1.1000,0.0000) phase=1.3500 mode=2 segment=1 patch_hits=1 guard=3.0000
t=15.0000 p=(15.0000,1.5000,0.0000) phase=1.7500 mode=2 segment=1 patch_hits=1 guard=7.0000

certified crossing: 1 interpolation=0.5000
fixed structural bound: segment tests=4, patch tests=4, seed evaluations=1
```

## Structural source audit

```
PASS: fixed MaxSegments/MaxPatches hot-query structure present
```

# Fresh literal GPU result

The optional CUDA target was compiled and physically executed for this delivery. It launches one independent query per CUDA thread, then compares every returned x position with the CPU result.

## GPU and toolchain

```
NVIDIA GeForce RTX 5070 Ti Laptop GPU, 591.59  
cmake version 4.3.2  
Build cuda_12.8.r12.8/compiler.35404655_0
```

## CUDA smoke executable

```
CUDA smoke test passed for 4096 queries.
```

**Result: the GPU agreed with the CPU for all 4,096 generated queries within the source tolerance of 1e-5.**

## What this does and does not prove

- It proves this source built and its included CPU test passed on this machine.
- It proves the CUDA smoke program ran on the named GPU and matched its CPU reference for 4,096 queries.
- It does not provide a throughput, latency, memory-bandwidth, or energy benchmark.
- It does not make all possible archives constant-cost. The  $O(1)$  claim depends on keeping the compile-time profile fixed and rejecting overflow.

## Plain-language accuracy card

| Question                       | Accurate answer                                                                                                                 |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Is one indexed query $O(1)$ ?  | Yes, under one fixed profile: exactly 4 segment tests, 4 patch tests, and 1 seed evaluation in this demo profile.               |
| Are $q$ answers $O(1)$ total?  | No. Producing $q$ outputs is $\Theta(q)$ .                                                                                      |
| Is runtime literally $o(1)$ ?  | No. Nontrivial work cannot shrink toward zero in the ordinary RAM/GPU model.                                                    |
| Where can little- $o$ apply?   | To encoded bytes divided by $N$ dense materialized samples, when novelty is fixed or sublinear as $N$ grows.                    |
| What if more slots are needed? | Reject, split, choose a larger named profile, or use a variable-cost representation. Do not silently keep the old $O(1)$ claim. |

## Implementation map

| Source step          | Meaning                                                                                       |
|----------------------|-----------------------------------------------------------------------------------------------|
| select_segment       | Scans the complete fixed segment array and chooses the matching half-open time interval.      |
| predict_seed         | Evaluates the fixed quadratic position and circular phase expression once.                    |
| evaluate_patch_slots | Scans the fixed patch array and applies the matching cumulative correction.                   |
| evaluate_guard       | Evaluates the plane or sphere function plus support, compatibility, and error-contract gates. |
| classify_event       | Reports a crossing only when endpoint signs and the interval certificate justify it.          |

Fresh execution note: the original packaged validation summary says native CUDA compilation and physical GPU execution were not performed at packaging time. This report does not rewrite that historical record; it adds a new local run in which both succeeded.